Distributed streaming collaboration

Technical explanation report - application implementation and integration status

Owner B handoff | 10 November 2026 | Base revision 4b67076

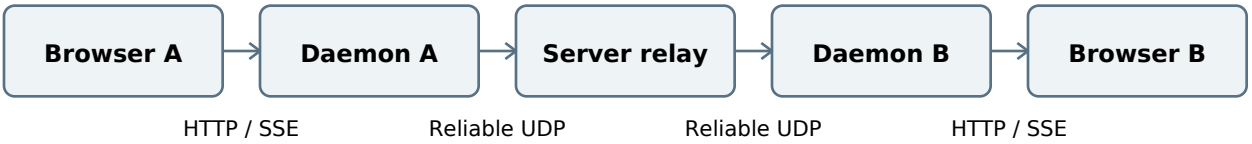
The application provides authenticated global/private chat and receiver-confirmed file transfers through a central server. This report covers the B implementation, its executable tests and its remaining dependencies. It is a draft for integration with A, not a final claim that all ten rubric steps pass.

Rubric coverage at handoff

Step	Evidence or dependency
1	Application schema/state supplement and sequences; A owns final transport specification.
2	B lifecycle hooks and stub ordering tests exist; A2 module is absent. Not complete.
3	Salted PBKDF2, registry IP/port, wrong/duplicate/repeated login tests.
4	20 localhost clients, all logged in before broadcast; exact delivery assertions.
5	Global/private routing; third-party isolation; file participant/state validation.
6	Zero/boundary/Unicode files and lossy two-hop relay pass. Large-file bounds await A5.
7	Crash/partial cleanup, reconnect, stop/logout and HTTP errors tested; LAN pending.
8	Existing A transport AES-GCM is used. Control security/key authentication await A6.
9	Custom HTTP/SSE, client-side JavaScript, streaming upload and status; browser QA pending.
10	Automated local tests and evidence; two A transport skips and physical LAN remain.

Sources: supplied Assignment 1 DOCX, sections 6-10 and rubric; WORKPLAN.md B1-B7; supplied CO3094 PDF pp. 7-12; instructor example repository SPKILLER2301/temp2.

Architecture and protocol



A2 TCP bootstrap precedes UDP when supplied; absent in current base revision.

Each user runs a local daemon and one browser tab. HTTP and SSE connect the browser to its daemon on loopback. The daemon uses Endpoint to contact the central server. Registry derives sender identity from the authenticated connection; Router selects recipients and allocates an independent outbound stream per relayed file.

Application messages

Stream	Messages	Validation
0	AUTH, LOGOUT, user events, FILE signals	Credentials and participant identity; state transitions and transfer limits.
1	MSG_PRIVATE, MSG_BROADCAST	Login required, text 1-4096 characters, private recipient required.
2 and above	FILE_BEGIN, binary chunks	Accepted offer, sender/receiver/size match, chunks up to 8192 bytes.

JSON messages are UTF-8 objects with a string type. File chunks are raw bytes. Metadata includes file_id, filename, integer size and SHA-256. File size is capped at 200 MiB; at most 16 active offers/transfers per participant. Filenames cannot contain path separators or NUL. Destination creation is exclusive and existing files are preserved.

Transfer state

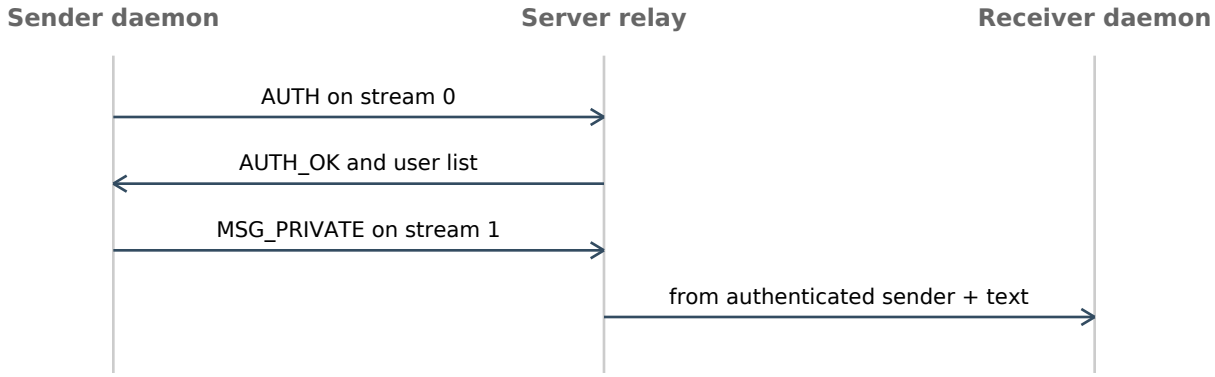
OFFERED becomes ACCEPTED only after the designated receiver answers. The sender then emits BEGIN, followed by chunks. Server state is RELAYING until the declared byte count is forwarded, then AWAIT_DONE. Only the designated receiver can return DONE; rejection, peer failure, disconnect or inactivity removes application state. Empty files skip chunks and verify the empty-file hash immediately.

Transport integration boundary

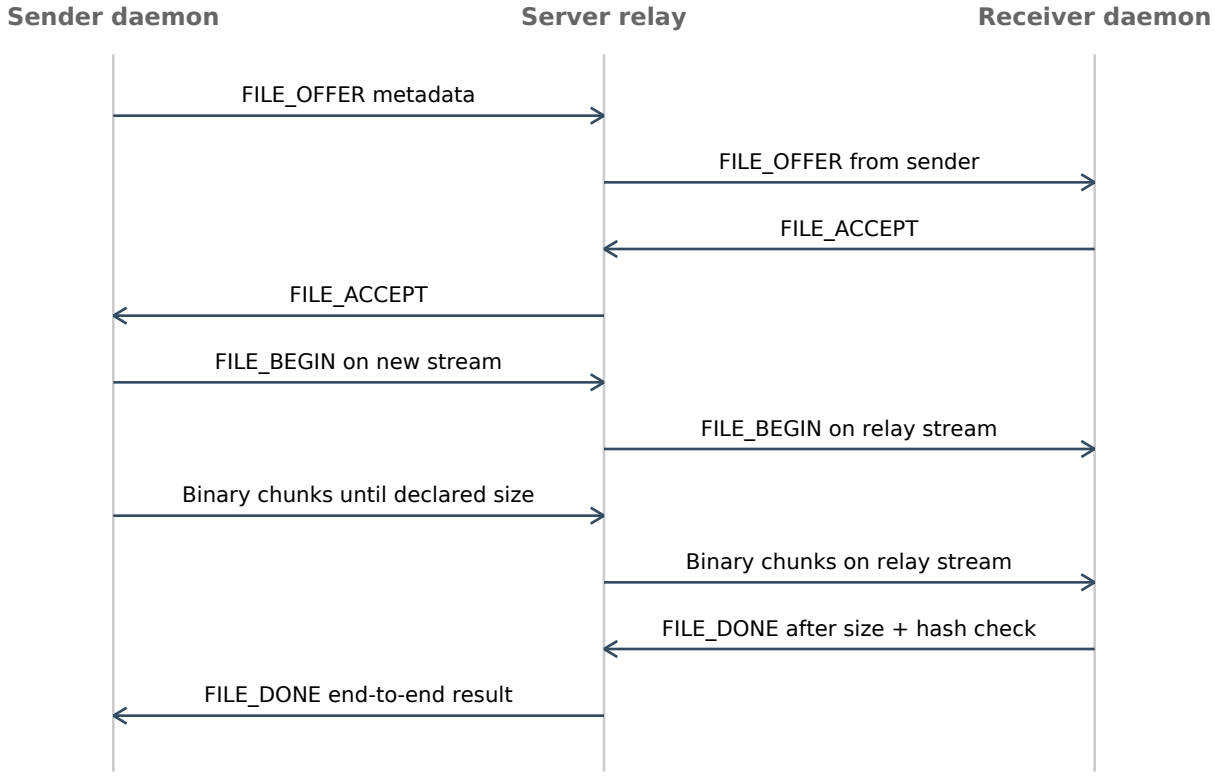
The current base uses protocol version 1, a 23-byte header and 1200-byte datagrams, with X25519/AES-GCM, per-stream sequence numbers and ACK/SACK recovery. These existing mechanisms belong to A; this report does not certify a complete QUIC-like transport. CUBIC, receive credit, bounded transport queues, control authentication and stream reset remain pending.

Message and file sequences

Authenticated private message



Accepted file



The receiver may return FILE_REJECT before BEGIN. Disconnect aborts the application transfer and deletes receiver partial output; there is no automatic resume. In v1, application cleanup does not yet reset transport buffers. Sender progress means queued bytes unless a real A5 receipt API is installed and validated.

Challenges and solutions

This page records problems identified in the code audit and the changes made in the B branch. Unimplemented transport work is listed as an integration dependency rather than described as a solved problem.

Accepting data without an agreed transfer

The initial router forwarded file signals without tracking who offered or accepted a file. BEGIN could create a relay without a matching accepted offer. An authoritative server transfer record now binds sender, receiver, size and state. Tests reject third-party acceptance, duplicate acceptance, premature DONE and chunks exceeding the declared size.

Empty files and incomplete output

The initial receiver completed only after a chunk write, so an empty file could never finish over the network. BEGIN now completes a zero-byte transfer immediately and verifies its hash. Partial output is deleted on errors and disconnect, and exclusive file creation prevents overwriting another transfer. Boundary tests include 0, 1, 8192 and 8193 bytes, Unicode names and disconnect during an active transfer.

Blocking authentication and stale identity

Bare SHA-256 records were replaced with salted PBKDF2 demo credentials. Verification runs outside the UDP receive callback using asyncio.to_thread. The server allows one pending login task per connection, tracks attempts and an authentication deadline, and prevents one connection from being bound to two identities. Same-account repeated login is idempotent; changing identity requires logout.

Uploads and progress semantics

The original HTTP handler read a complete upload body into memory and reused the submitted filename in one temporary directory. The handler now reads at most 64 KiB at a time into a unique upload directory, applies per-request limits/timeouts and cleans truncated uploads. Tests cover fragmented input and simultaneous same-name files. UI progress distinguishes enqueue from server ACK and waits for receiver DONE before declaring success.

Recovery and unresolved transport pressure

One reconnect supervisor uses backoff and jitter, reauthenticates with memory-only credentials and is cancelled by logout/stop. Active files abort rather than resume. HTTP UI starts before network startup so an unavailable server does not hide the UI. However, yielding the event loop is not backpressure: v1 queues remain unbounded. B cannot claim large-file memory safety until A5 supplies bounded send/receive APIs and the relay switches to async receive/consume.

Evaluation and remaining work

Observed localhost evidence

Scenario	Result and scope
Baseline	22 passed, 4 skipped on unchanged main at 4b67076.
B implementation	See docs/evidence/B-application.md and pytest-B-final.txt for final recorded count.
20 clients	380 expected / 380 received / 0 duplicates. Barrier before first broadcast.
Observed timings	Example run: logins 0.228 s; complete measurement 0.612 s. Machine-specific; not a performance guarantee.
Memory	Example peak traced Python allocations: 1,293,111 bytes in the load generator, excluding server RSS.
Lossy file relay	32 KiB payload, 10% loss, 1 ms delay + 0-3 ms jitter and 2% duplicate probability on each endpoint outbound path; seeds 31/32/33; receiver hash and sender DONE checked.
HTTP	Offline state, invalid request/body, traversal/origin rejection, fragmented uploads, duplicate names, truncated upload cleanup and one-tab SSE.

The suite is automated on Linux/Python 3.12. The remaining A transport skips are HoL isolation and peer-disconnect detection. B has a separate end-to-end crash-cleanup test; this does not implement those A placeholders. Large 10/100 MB tests, bounded receive/relay memory, LAN on two physical hosts, Windows/macOS and manual browser visual QA have not been completed.

Before final submission

A supplies bootstrap, bounded async send/receive and cancellation/stats interfaces, CUBIC and transport-security validation. B then completes async relay integration and real ACK progress. Both members run full startup/reconnect through TCP and UDP, slow-receiver/large-file tests, loss and LAN trials, then merge final protocol/security sections and evidence into the report.

Source interpretation

Assignment 1 specifies central server routing and UDP files. The supplemental CO3094 PDF describes raw TCP, HTTP authentication and direct P2P/channel functionality, while p. 12 points grading back to Assignment 1. The example contains async, standard-socket and raw-TCP branches and unfinished parsing/routing TODOs. These materials do not establish an instructor-approved resolution of every discrepancy.

Source links: github.com/SPKILLER2301/Distributed-Streaming-based-Client-Server (WORKPLAN.md and source); github.com/SPKILLER2301/temp2 (CO3094_streaprous/daemon and apps). Full comparison and handoff: docs/AUDIT_B.md. Course example source was reviewed, not copied into this package.